CampusHub 项目 AI 工具链选型原因

团队名称： 倒地爆冲

本团队在综合评估了项目需求与各 AI 工具的特性后，决定采用**“多工具互补组合”**的策略，以应对敏捷开发不同阶段的复杂场景。具体选型与评估如下：

1. AI 编程助手 (评估候选：Cursor, Antigravity)

- **最终决策：两者结合使用**
- **选择 Cursor 的理由**：
  - **场景分工**：用于从零搭建核心模块或进行大规模代码重构。
  - **评估**：Cursor 作为深度定制的 AI IDE，其全局代码库理解能力（Codebase Context）极为强大。在 P4 核心编码阶段，它能极大提升整文件生成和复杂逻辑编写的效率。
- **选择 Antigravity 的理由**：
  - **场景分工**：用于日常轻量级开发、快速查错与局部代码补全。
  - **评估**：作为轻量级插件，它能无缝集成到团队现有的 VS Code 环境中。易用性极高，团队成员无需改变原有习惯，适合在修改小 Bug 或查阅 API 文档时快速唤起。

2. AI 对话工具 (评估候选：Gemini, ChatGPT)

- **最终决策：两者结合使用**
- **选择 Gemini 的理由**：
  - **场景分工**：主攻 P1 需求分析阶段的长文档处理与架构设计辅助。
  - **评估**：Gemini 拥有超大的上下文窗口（Context Window）。我们可以在需求分析阶段，将所有的用户调研问卷、访谈记录以及 IEEE 830 标准要求一次性喂给它，它不会遗忘上下文，非常适合生成长篇文档。
- **选择 ChatGPT 的理由**：
  - **场景分工**：主攻 P3 详细设计与 P4 编码阶段的逻辑推导与 Debug。
  - **评估**：ChatGPT 在代码逻辑推理、算法设计和报错信息解析方面表现卓越。当遇到复杂的业务逻辑（如订单状态机流转）或死锁报错时，其强大的逻辑思辨能力能提供最准确的解决方案。

3. AI 代码审查工具 (评估候选：CodeRabbit, AI Code Review)

- **最终决策：两者结合使用**
- **选择 CodeRabbit 的理由**：
  - **场景分工**：作为 GitHub 仓库的主力 PR (Pull Request) 审查员。
  - **评估**：开箱即用，集成成本极低。团队成员每次发起合并请求时，CodeRabbit 会自动进行逐行代码审查，并给出优化建议，大幅减轻团队“开发负责人”的人工 Review 负担。
- **选择 AI Code Review (脚本/Action类) 的理由**：
  - **场景分工**：作为 CI 流水线中的底层安全与规范拦截器。
  - **评估**：相比于 CodeRabbit 的业务向审查，我们可以在 CI 中配置定制化的 AI Code Review 脚本，专门针对团队特定的代码规范（如变量命名规范、硬编码密码检查）进行硬性拦截。

4. AI 测试工具 (评估候选：CodiumAI, 通义灵码)

- **最终决策：选择 CodiumAI**

- **选择 CodiumAI 的理由**：
  - **场景分工**：用于 P6 质量保障阶段生成单元测试用例。
  - **评估**：它是专门针对测试场景优化的 AI 工具，不仅能自动生成覆盖正常流的测试，还能主动分析代码并生成边界条件（Edge Cases）的测试用例，功能专一且强大。
- **不选择通义灵码的理由**：
  - 虽然其功能全面，但我们已经有了 Cursor 和 Antigravity 作为编程助手，为了避免工具链过于臃肿，我们倾向于在测试环节使用垂直领域更专业的 CodiumAI。

角色分工阐述

1. 需求负责人 (主导 P1 阶段：第 2-3 周)

- **对角色的理解**：
  - 需求负责人是项目的“产品经理”，负责定义 CampusHub “做什么”以及“为谁做”。
  - 在 AI 时代，该角色不仅要收集需求，更要识破 AI 生成的“虚假需求”或“逻辑幻觉”。
- **核心职责与目标**：
  - **调研与获取**：设计并执行用户调研方案，确保收集至少 20 份有效问卷和 8 人次深度访谈笔记。
  - **AI 盲区挑战**：主导“AI 需求盲区挑战”实验，对比 AI 盲写的需求与人工调研结果，识别至少 3 处 AI 的遗漏或捏造点 。
  - **标准化建模**：按照 IEEE 830 标准撰写需求规格说明书 (SRS) 。编写至少 15 条包含验收标准的用户故事，并使用 MoSCoW 方法进行优先级排序。
  - **核心目标**：确保 P0 级核心功能（用户管理、需求发布、订单流程）的业务逻辑无歧义，为后续开发打下地基。

2. 架构负责人 (主导 P2 阶段：第 4 周)

- **对角色的理解**：
  - 架构负责人是系统的“总设计师”，负责决定项目的技术底座和数据流向。
  - 其重点在于平衡功能实现与系统质量（如安全性、性能）。
- **核心职责与目标**：
  - **技术选型与 ADR**：负责确定前后端分离的技术选型（已定为 Vue 3 + Node.js + MySQL）并撰写架构决策记录 (ADR)。
  - **非功能性需求设计**：设计系统的安全性方案（如密码加密存储、防止 SQL 注入）以及性能方案（确保页面加载 < 2 秒）。
  - **基建维护**：主导 P0 阶段的 Git 仓库搭建、分支策略制定（main/dev/feature）以及 CI 流水线的自动化配置。
  - **核心目标**：构建一个高内聚、低耦合的系统骨架，确保代码结构清晰，具备良好的可维护性。

3. 开发负责人 (主导 P4 阶段：第 6-7 周)

- **对角色的理解**：
  - 开发负责人是代码的“首席执行官”，负责将设计转化为高质量的运行系统。
  - 在本项目中，该角色还要监督《AI 协作契约》的落地，防止团队过度依赖 AI 导致能力退化 。
- **核心职责与目标**：
  - **规范制定**：确立团队编码规范，并监督所有 Commit 消息是否按规定标注 `[AI-assisted]` 或 `[Human-written]`。

- **AI 代码审查**：主导 Code Review 流程，确保所有 AI 生成的代码必须经过至少一名成员的人工审查后方可合入。
- **核心模块开发**：主导 P0 级核心功能的编码，并确保每位成员在每个编码阶段至少手写 2 个核心函数。
- **核心目标**：在 10 周内交付一个功能完整、无重大逻辑漏洞的系统，并确保 Prompt 日志记录完整。

4. 测试负责人 (主导 P6 阶段：第 9 周)

- **对角色的理解**：
  - 测试负责人是项目的“质量守门员”，负责从用户和工程角度验证系统是否“做对了”。
- **核心职责与目标**：
  - **测试用例设计**：为至少 3 个核心用例编写详细的测试描述，涵盖前置条件、基本流及各种异常替代流。
  - **质量保障监控**：利用 AI 工具辅助生成单元测试用例，确保核心模块具备基本的单元测试覆盖。
  - **终期评审准备**：负责 P6 阶段的质量报告撰写，检查系统是否满足“核心操作路径不超过 3 步”的可用性要求 。
  - **核心目标**：通过全方位的测试，将系统 Bug 率降至最低，确保项目在 P7 阶段顺利答辩并交付。